



2026

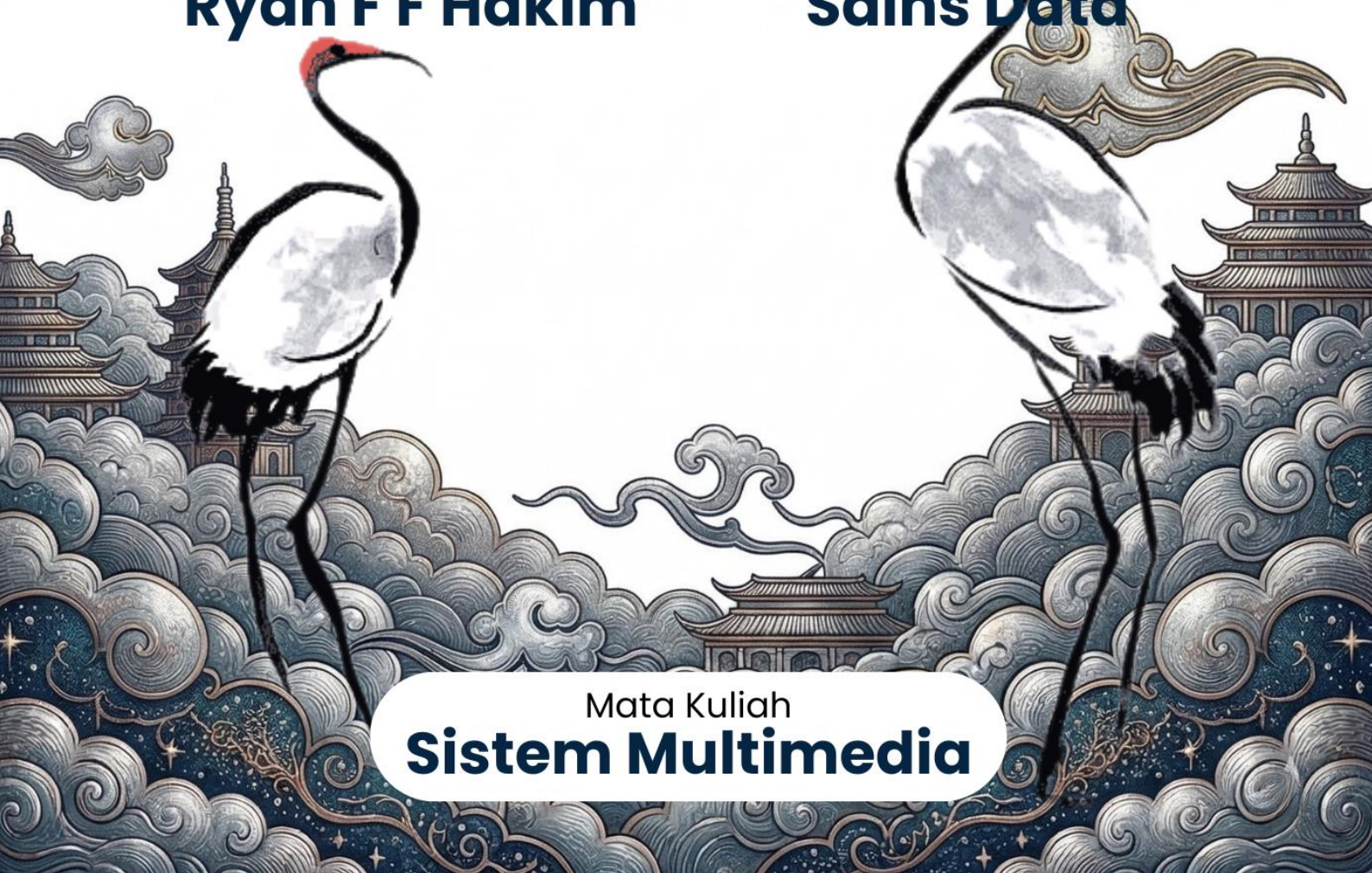
MODUL PRAKTIKUM 3
BASIC DIGITAL IMAGE PROCESSING
OPTIMIZED VERSION

Disusun Oleh:

Ryan F F Hakim

Dipersembahkan Untuk:

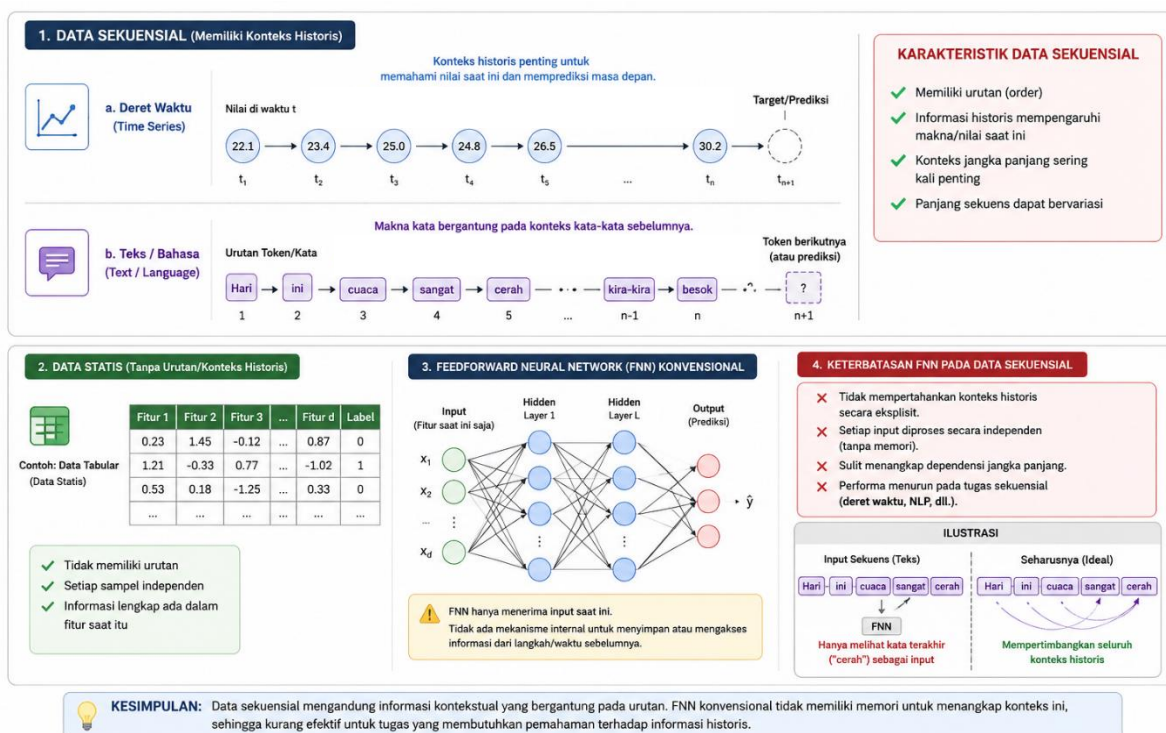
Sains Data



Mata Kuliah
Sistem Multimedia

PENDAHULUAN

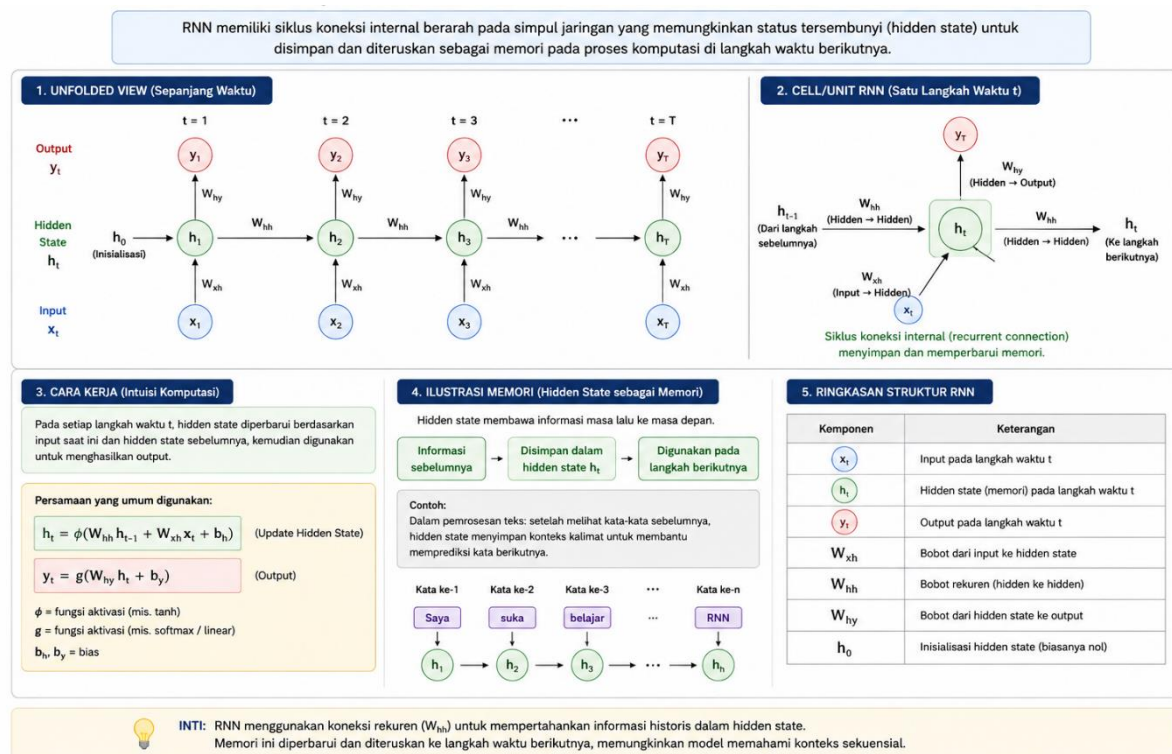
Dalam era perkembangan kecerdasan buatan, berbagai arsitektur jaringan saraf tiruan telah dikembangkan untuk menyelesaikan masalah komputasi yang kompleks. Salah satu tantangan utama yang dihadapi adalah pemrosesan data sekuensial, di mana urutan informasi memiliki makna yang sangat penting, seperti pada teks, suara, maupun data deret waktu (*time series*). Pemrosesan jenis data ini tidak dapat dilakukan secara optimal oleh jaringan saraf tiruan konvensional (*Feedforward Neural Network*) karena ketidakmampuannya dalam menyimpan informasi dari urutan waktu sebelumnya. Oleh karena itu, sebuah arsitektur khusus sangat dibutuhkan dan dirancang agar konteks waktu serta urutan data dapat dipertahankan dan dianalisis dengan baik.



Gambar 1.1 Visualisasi karakteristik data sekuensial, seperti deret waktu dan teks, yang dikontraskan dengan data statis untuk menunjukkan keterbatasan arsitektur Feedforward Neural Network (FNN) konvensional dalam mempertahankan dan memproses konteks informasi historis.

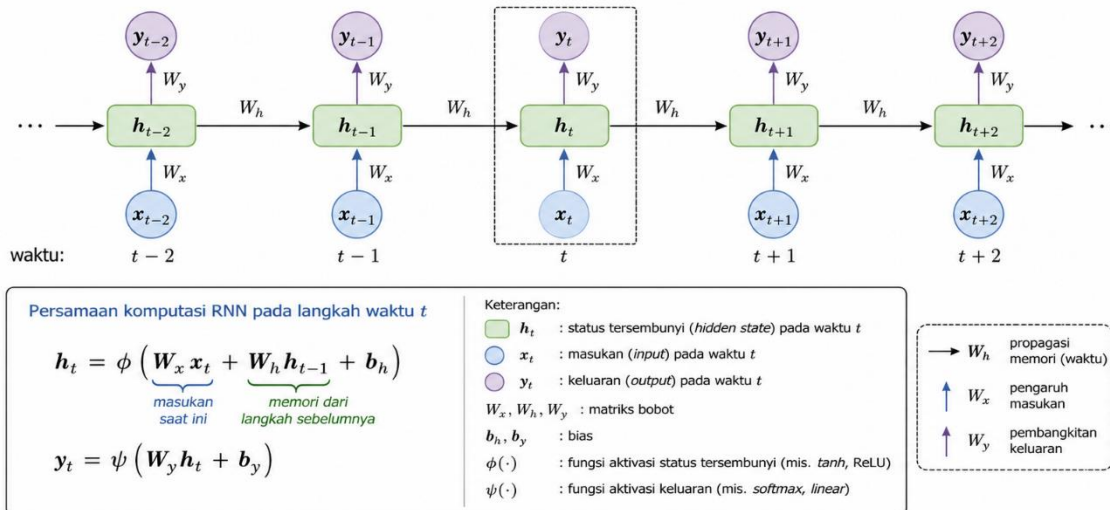
Untuk mengatasi keterbatasan tersebut, arsitektur *Recurrent Neural Network* (RNN) diperkenalkan dan diaplikasikan secara luas dalam ranah pembelajaran mendalam (*deep learning*). RNN diartikan sebagai sebuah kelas jaringan saraf tiruan di mana koneksi antar simpul membentuk siklus berarah yang sejalan dengan urutan waktu. Melalui struktur komputasi ini, sebuah kondisi memori internal (*hidden state*) dapat dibentuk sehingga informasi dari masukan sebelumnya dapat diproses dan dipertahankan untuk memengaruhi keluaran pada langkah waktu saat ini. Kemampuan mengingat informasi masa lalu ini menjadikan arsitektur RNN sangat

ideal dan banyak digunakan dalam berbagai aplikasi pemrosesan bahasa alami (*Natural Language Processing*) dan peramalan (*forecasting*).



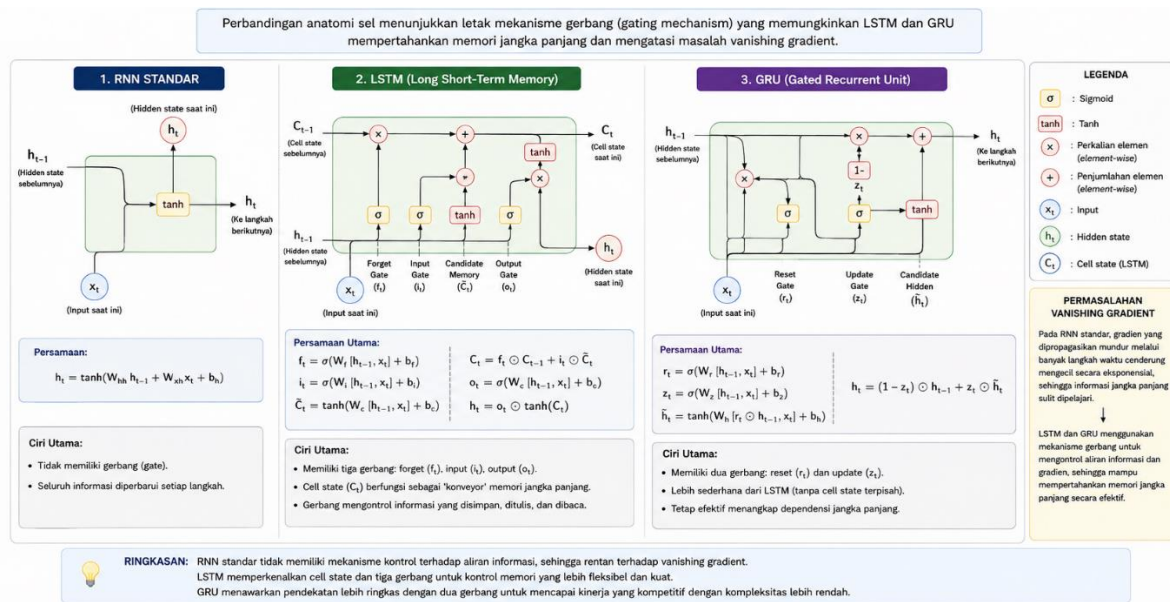
Gambar 1.2 Representasi struktural dari arsitektur dasar Recurrent Neural Network (RNN), menampilkan siklus koneksi internal berarah pada simpul jaringan yang memungkinkan status tersembunyi (hidden state) untuk disimpan dan diteruskan sebagai memori pada proses komputasi di langkah waktu berikutnya.

Secara mekanisme, proses komputasi pada RNN dijalankan dengan cara menerima masukan yang tidak hanya berasal dari data pada langkah waktu saat ini, melainkan juga dari status tersembunyi yang dihasilkan pada langkah waktu sebelumnya. Informasi yang terkandung di dalam urutan data ini digabungkan secara matematis melalui fungsi aktivasi untuk menghasilkan keluaran baru sekaligus memperbarui status memori. Siklus komputasi yang diterapkan secara berulang pada setiap elemen sekuens ini memungkinkan jaringan untuk menangkap pola dinamis dan dependensi temporal yang tersembunyi di dalam struktur data.



Gambar 1.3 Skema penjabaran waktu (*unrolled in time*) dari proses komputasi RNN, yang mengilustrasikan secara matematis bagaimana masukan pada langkah waktu saat ini (x_t) diintegrasikan dengan status tersembunyi dari langkah sebelumnya (h_{t-1}) untuk menghasilkan keluaran (y_t) dan memperbarui memori jaringan secara iteratif.

Meskipun memiliki keunggulan teoretis yang kuat, kendala teknis sering kali ditemui saat RNN standar dilatih pada data dengan rentang sekuens yang sangat panjang. Permasalahan matematis seperti gradien yang menghilang (*vanishing gradient*) atau meledak (*exploding gradient*) menyebabkan kesulitan besar bagi jaringan untuk mempelajari dependensi jangka panjang. Sebagai solusi atas permasalahan tersebut, varian RNN yang lebih mutakhir seperti *Long Short-Term Memory* (LSTM) dan *Gated Recurrent Unit* (GRU) telah dikembangkan dan diimplementasikan secara ekstensif. Arsitektur-arsitektur lanjutan ini dilengkapi dengan mekanisme gerbang (*gating*) yang dikontrol sedemikian rupa untuk mengatur aliran informasi, sehingga memori jangka panjang dapat disimpan, diperbarui, atau dibuang sesuai dengan relevansinya.



Gambar 1.4 Diagram perbandingan anatomi sel antara RNN standar, Long Short-Term Memory (LSTM), dan Gated Recurrent Unit (GRU), menonjolkan letak mekanisme gerbang pengontrol (gating mechanism) yang dirancang khusus untuk mempertahankan memori jangka panjang dan mengatasi masalah gradien yang menghilang (vanishing gradient).

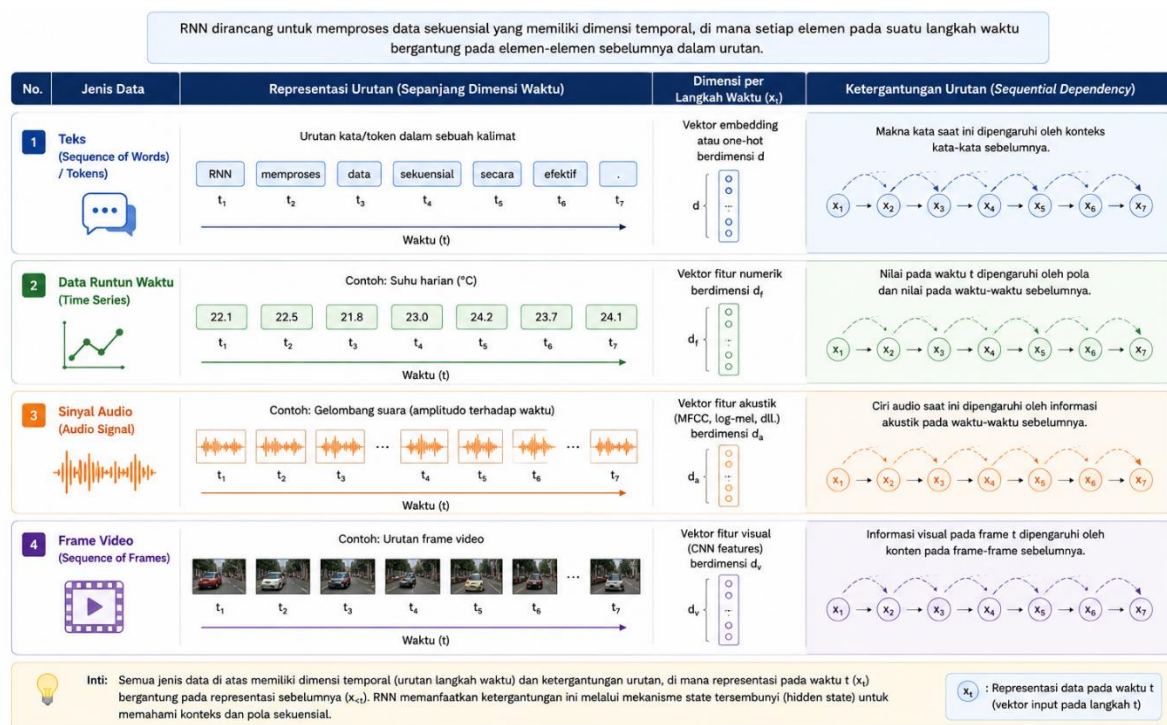
Melalui modul praktikum ini, pemahaman komprehensif mengenai konsep dasar, arsitektur, dan cara kerja dari *Recurrent Neural Network* akan diberikan secara terstruktur. Implementasi model RNN beserta variannya akan dilakukan secara langsung menggunakan bahasa pemrograman dan pustaka pembelajaran mesin terkini. Selain itu, tahapan fundamental seperti prapemrosesan data sekuensial, perancangan arsitektur jaringan, proses pelatihan model, hingga evaluasi performa pada studi kasus nyata akan dipraktikkan secara sistematis. Dengan diselesaikannya modul ini, penguasaan keterampilan teknis yang mumpuni dalam merancang dan mengaplikasikan algoritma RNN untuk memecahkan masalah berbasis data sekuensial diharapkan dapat dicapai oleh para praktikan.

LANDASAN TEORI

2.1 RECURRENT NEURAL NETWORK (RNN)

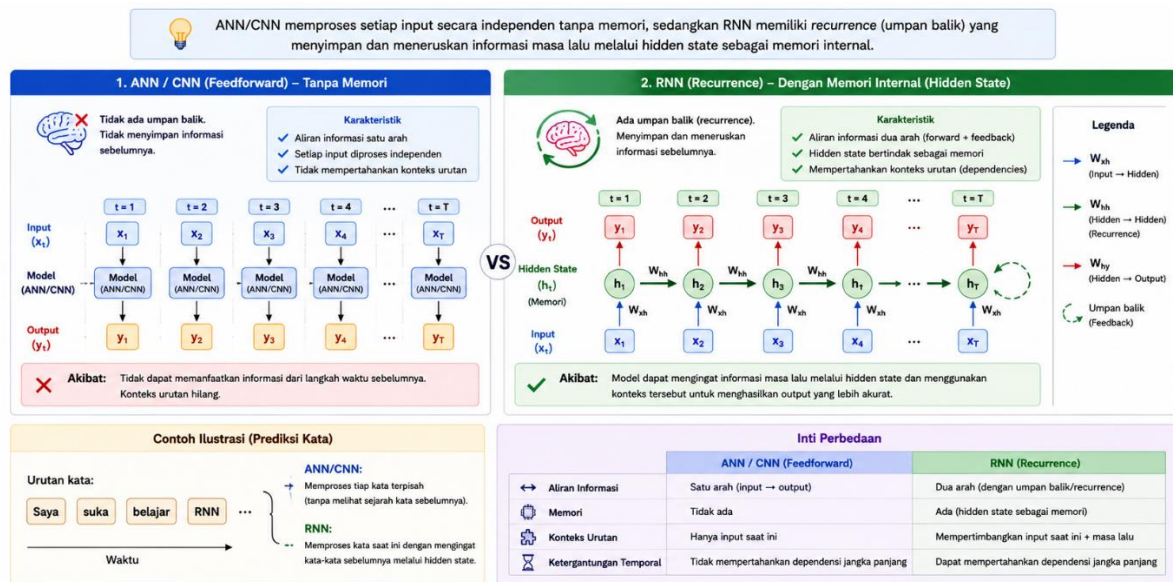
Recurrent Neural Network (RNN) merupakan kelas arsitektur *deep learning* yang didesain secara spesifik untuk memproses **data sekuensial**. Berbeda dengan arsitektur standar yang mengasumsikan setiap titik data bersifat independen, data sekuensial memiliki ketergantungan temporal atau struktural di mana urutan data sangatlah krusial. Contoh data sekuensial meliputi:

- **Teks (Natural Language):** Rangkaian kata yang membentuk kalimat dan konteks.
- **Data Runtun Waktu (Time Series):** Perubahan nilai seiring berjalannya waktu (misalnya harga saham, data sensor IoT, dan cuaca).
- **Sinyal Audio:** Rangkaian gelombang suara dalam dimensi waktu.
- **Video:** Urutan *frame* citra (data spatiotemporal).



Gambar 2.1: Representasi dimensi temporal dan ketergantungan urutan (sequential dependency) pada berbagai jenis data, meliputi teks, data runtun waktu (time series), sinyal audio, dan frame video yang menjadi basis pemrosesan arsitektur Recurrent Neural Network (RNN).

Pada Artificial Neural Network (ANN) standar, data diproses tanpa menyimpan konteks dari input sebelumnya. Convolutional Neural Network (CNN) menangkap informasi spasial dengan sangat baik, namun tidak memiliki mekanisme memori bawaan untuk melacak urutan temporal. Kendala ini diatasi oleh RNN dengan menggunakan mekanisme **recurrence** (pengulangan). Mekanisme ini mampu menyimpan memori dari langkah-langkah sebelumnya untuk memengaruhi prediksi pada langkah saat ini.



Gambar 2.2: Perbandingan konseptual aliran informasi antara ANN/CNN (feedforward) yang memproses data secara independen, dengan RNN yang memiliki mekanisme recurrence (umpan balik) sebagai memori internal untuk melacak urutan temporal.

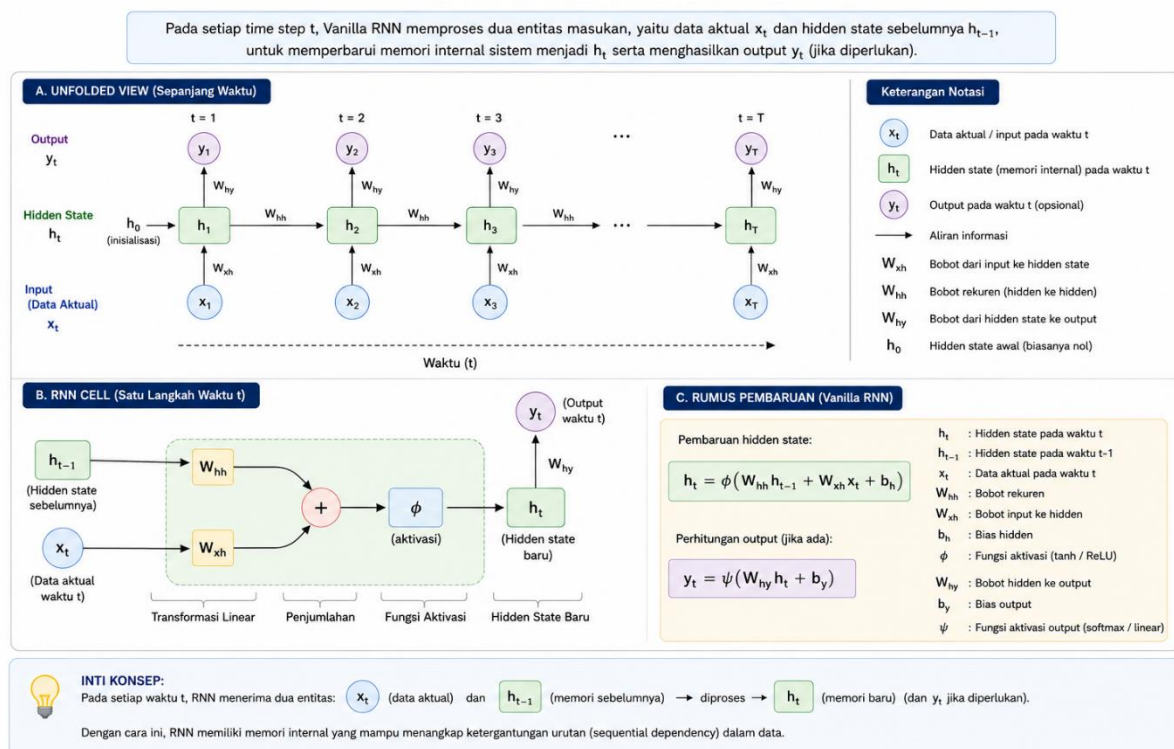
Perbandingan Karakteristik Arsitektur

Aspek	ANN (Fully Connected)	CNN (Convolutional)	RNN (Recurrent)
Karakteristik Masukan	Vektor ukuran tetap dan independen	Grid spasial (citra 2D/3D)	Sekuensial dengan panjang bervariasi
Pola Ekstraksi	Pemetaan linear tanpa urutan	Ekstraksi fitur lokal dan hierarkis	Ketergantungan temporal (urutan)
Mekanisme Memori	Tidak ada	Terbatas pada batas <i>receptive field</i>	Ada (<i>hidden state</i> meneruskan memori)

2.2 ARSITEKTUR DASAR RECURRENT NEURAL NETWORK (RNN)

Arsitektur RNN dasar, yang sering disebut sebagai *Vanilla RNN*, memproses sekumpulan data secara berurutan pada setiap satuan titik waktu (*time step*). Kemampuan memori temporal pada RNN difasilitasi oleh **hidden state** (h_t) yang bertindak sebagai memori internal. Pada setiap satuan waktu t , dua entitas masukan diproses secara bersamaan oleh RNN:

1. Data pada waktu saat ini (x_t).
2. *Hidden state* dari langkah sebelumnya (h_{t-1}).



Gambar 2.3: Arsitektur dasar *Vanilla RNN* yang menunjukkan pemrosesan dua entitas masukan pada setiap *time step* (t), yaitu data aktual (x_t) dan *hidden state* sebelumnya (h_{t-1}) untuk memperbarui memori internal sistem (h_t).

2.2.1 DEFINISI MATEMATIS OPERASI RNN

Secara matematis, pembaruan *state* pada RNN diformulasikan ke dalam transformasi linear yang kemudian dilewatkan pada fungsi aktivasi non-linear:

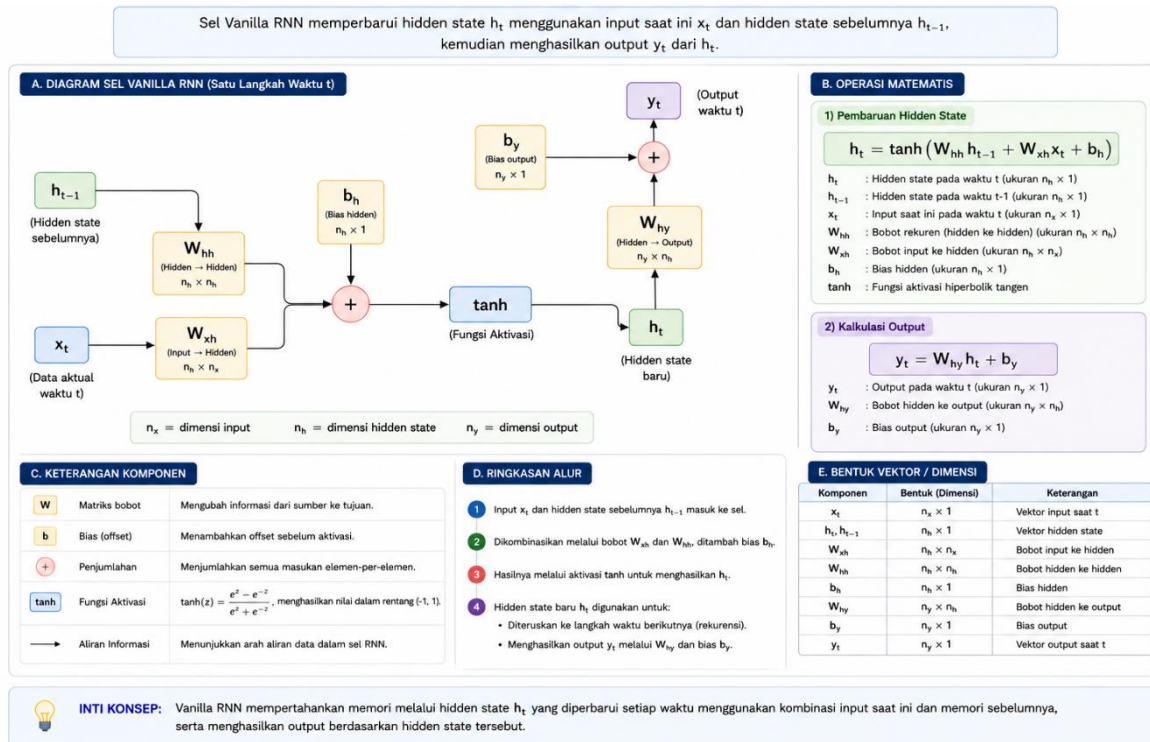
$$h_t = \tanh(W_{xh} \cdot x_t + W_{hh} \cdot h_{t-1} + b_h)$$

Sedangkan, probabilitas atau proyeksi target (*output*) dari jaringan dihitung melalui persamaan regresi:

$$y_t = W_{hy} \cdot h_t + b_y$$

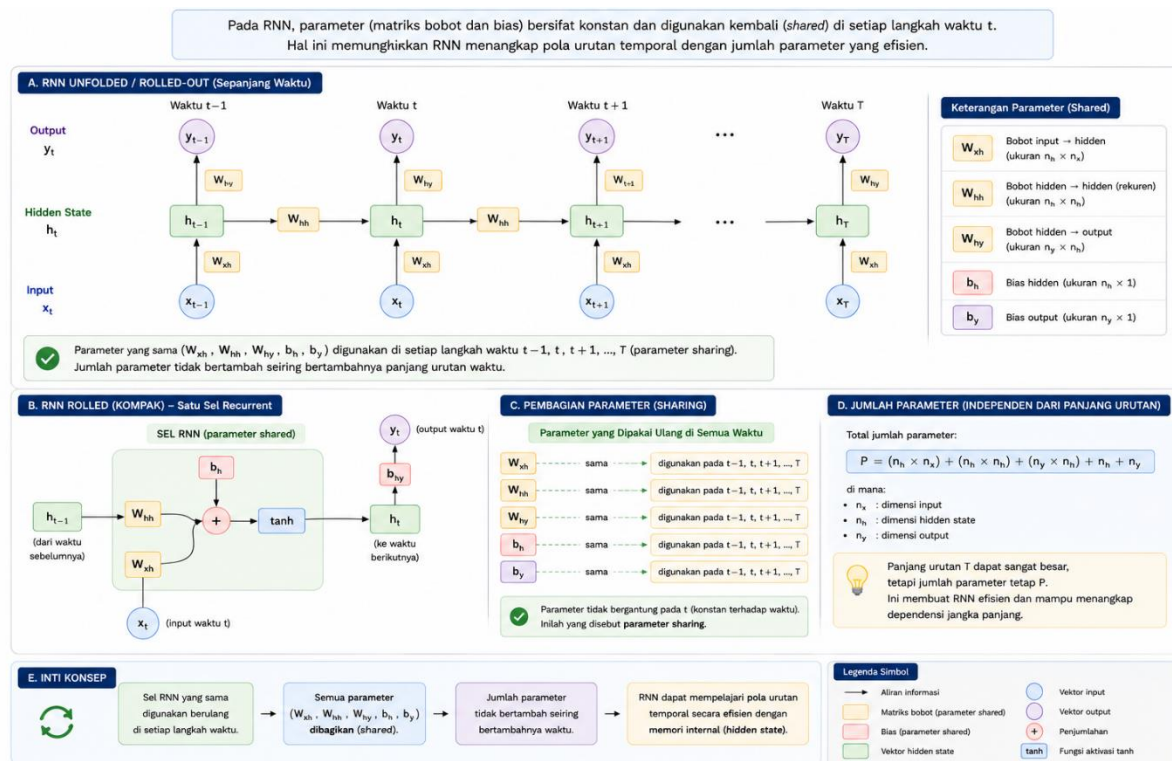
Keterangan parameter matematis:

- x_t : Vektor input pada langkah waktu t .
- h_t : Vektor *hidden state* pada waktu t , yang juga akan diteruskan sebagai ingatan ke langkah waktu berikutnya ($t + 1$).
- h_{t-1} : Vektor memori *hidden state* dari langkah waktu sebelumnya.
- W_{xh} : Matriks bobot untuk input (*input-to-hidden*).
- W_{hh} : Matriks bobot untuk siklus internal (*hidden-to-hidden*).
- W_{hy} : Matriks bobot untuk proyeksi output (*hidden-to-output*).
- b_h, b_y : Komponen bias.
- $\tanh$: Fungsi aktivasi non-linear yang membatasi nilai pada rentang absolut $[-1,1]$.



Gambar 2.4: Diagram sel *Vanilla* RNN yang mengilustrasikan operasi matematis pembaruan *hidden state* (h_t) dan kalkulasi luaran (y_t), lengkap dengan posisi matriks bobot (W_{xh}, W_{hh}, W_{hy}), bias (b_h, b_y), serta fungsi aktivasi *tanh*.

Konsep Parameter Sharing: Efisiensi dari arsitektur RNN didasarkan pada matriks pembobot (W_{xh}, W_{hh}, W_{hy}) yang dibagikan (**shared parameter**) pada setiap satuan waktu. Konvensi ini digunakan untuk meminimalisasi redundansi parameter sehingga jaringan fleksibel dalam menerima panjang urutan masukan yang bervariasi.



Gambar 2.5: Ilustrasi konsep *Parameter Sharing* pada RNN melalui representasi sel yang dijalankan (*unfolded/rolled-out*). Terlihat bahwa matriks bobot (W_{xh} , W_{hh} , W_{hy}) bersifat konstan dan digunakan kembali secara universal di setiap langkah waktu ($t-1, t, t+1$).

2.3 KENDALA PADA RNN TRADISIONAL: VANISHING GRADIENT

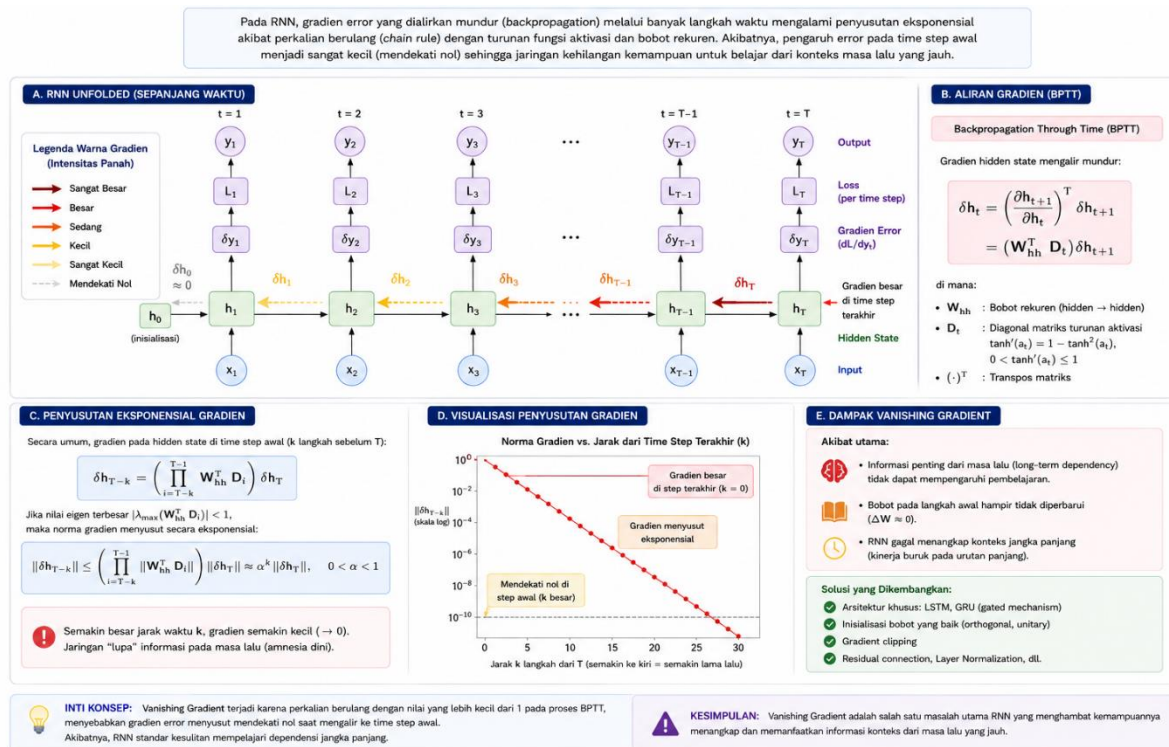
Dalam implementasi praktisnya, *Vanilla RNN* dihadapkan pada masalah kalkulus yang membatasi kemampuannya dalam mengingat informasi dari waktu yang jauh di masa lalu. Masalah ini lebih dikenal dengan istilah *long-term dependencies*.

Distribusi gradien error dalam arsitektur temporal dihitung melintasi rentang waktu menggunakan metode *Backpropagation Through Time* (BPTT). Sesuai dengan aturan rantai (*chain rule*) pada turunan, gradien error dikalikan berulang kali seiring perhitungan yang berjalan mundur ke langkah waktu sebelumnya.

Karena operasi turunan tersebut melibatkan perkalian matriks secara berulang, nilai turunan dapat menyusut secara eksponensial mendekati angka nol, memicu fenomena optimasi **Vanishing Gradient**. Indikasi terjadinya masalah ini meliputi:

- Terhentinya pembaruan bobot pada langkah waktu di masa lalu, karena gradien yang terlalu kecil.
- Jaringan kehilangan konteks dari awal informasi sekuensial.

- Operasional *Vanilla RNN* terbatas pada kemampuan ingatan sekitar 10 hingga 20 langkah waktu saja.



Gambar 2.6: Ilustrasi fenomena *Vanishing Gradient* pada proses *Backpropagation Through Time* (BPTT). Gradien *error* (diwakili oleh intensitas warna panah) menyusut secara eksponensial mendekati nol saat mengalir mundur ke *time step* awal akibat perkalian berulang (*chain rule*), menyebabkan jaringan mengalami "amnesia dini" terhadap konteks masa lalu.

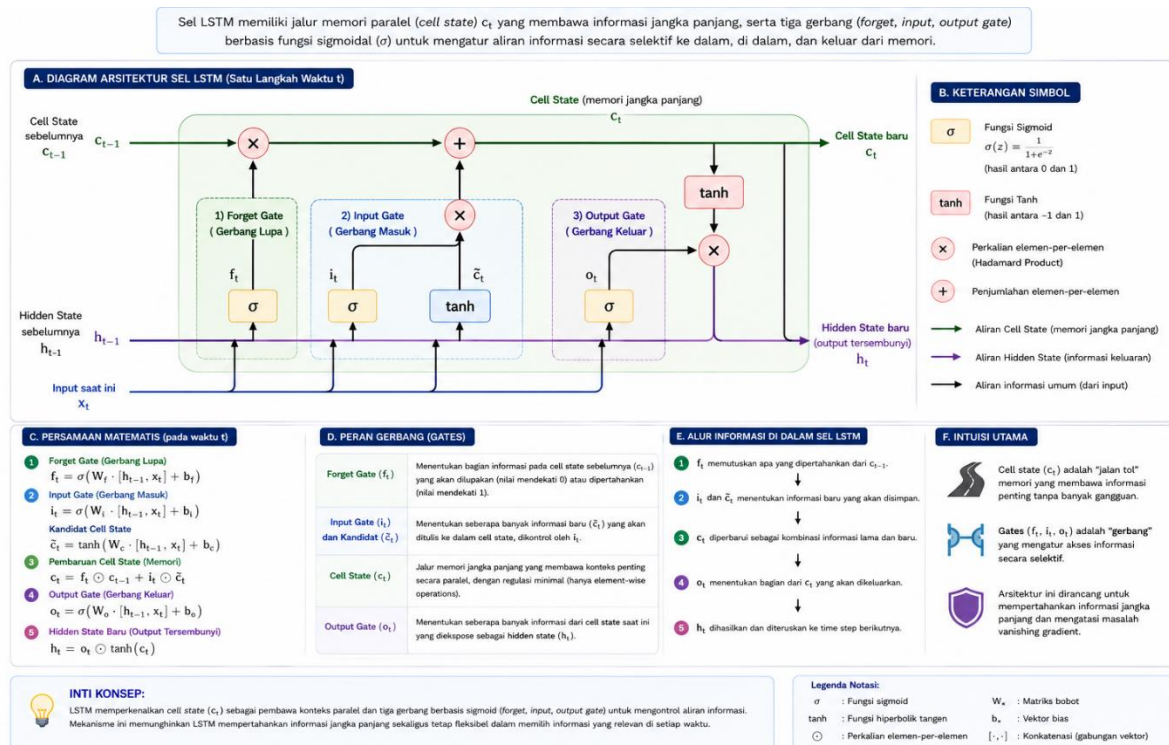
2.4 ARSITEKTUR LONG SHORT-TERM MEMORY (LSTM)

Untuk mengatasi hilangnya informasi akibat *vanishing gradient*, sebuah arsitektur baru diperkenalkan bernama **Long Short-Term Memory (LSTM)**. Keunikan dari topologi ini adalah digunakannya pembawa informasi secara tersendiri yang disebut **Cell State** ($\$c_t\$$). Jalur informasi ini diatur secara terkendali melalui komponen yang disebut sebagai algoritma "Gates" (Gerbang) dengan menggunakan fungsi sigmoid.

2.4.1 MEKANISME CELL STATE DAN OPERASI GATES

Alur informasi utama pada *Cell State* diibaratkan seperti ban berjalan (*conveyor belt*) yang membentang melintasi seluruh modul. Informasi dapat melintas dengan hanya sedikit perubahan matematis. Modifikasi informasi ini dikendalikan secara akurat oleh 3 gerbang berbasis *sigmoid* (σ) yang menghasilkan skala probabilitas pada rentang nilai $[0,1]$:

1. **Forget Gate (f_t):** Digunakan untuk menghitung fraksi informasi dari *Cell state* masa lalu yang selayaknya dilupakan atau dibuang.
2. **Input Gate (i_t):** Digunakan untuk mengatur porsi penambahan informasi baru yang akan diadaptasi menjadi matriks representasi terbaru (*candidate addition*).
3. **Output Gate (o_t):** Digunakan untuk menyelaraskan proporsi informasi mana saja dari *Cell state* yang relevan untuk dilepaskan pada waktu saat ini sebagai keluaran atau *hidden state*.



Gambar 2.7: Aliran komputasi internal sel LSTM yang menunjukkan jalur linear *Cell State* (seperti ban berjalan) dan mekanisme regulasi informasi oleh tiga gerbang berbasis sigmoid: *Forget Gate* (f_t), *Input Gate* (i_t), dan *Output Gate* (o_t).

2.4.2 FORMULASI MATEMATIS INTI LSTM

Satu kali iterasi arsitektur LSTM menjabarkan komputasi ke dalam empat tahapan berantai:

1. Kalkulasi Regresi Forget Gate:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

2. Kalkulasi Input Gate dan Validasi Modifikasi Kandidat (*Candidate*):

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{c}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$$

3. Eksekusi Pembaruan Lintasan Cell State Utama:

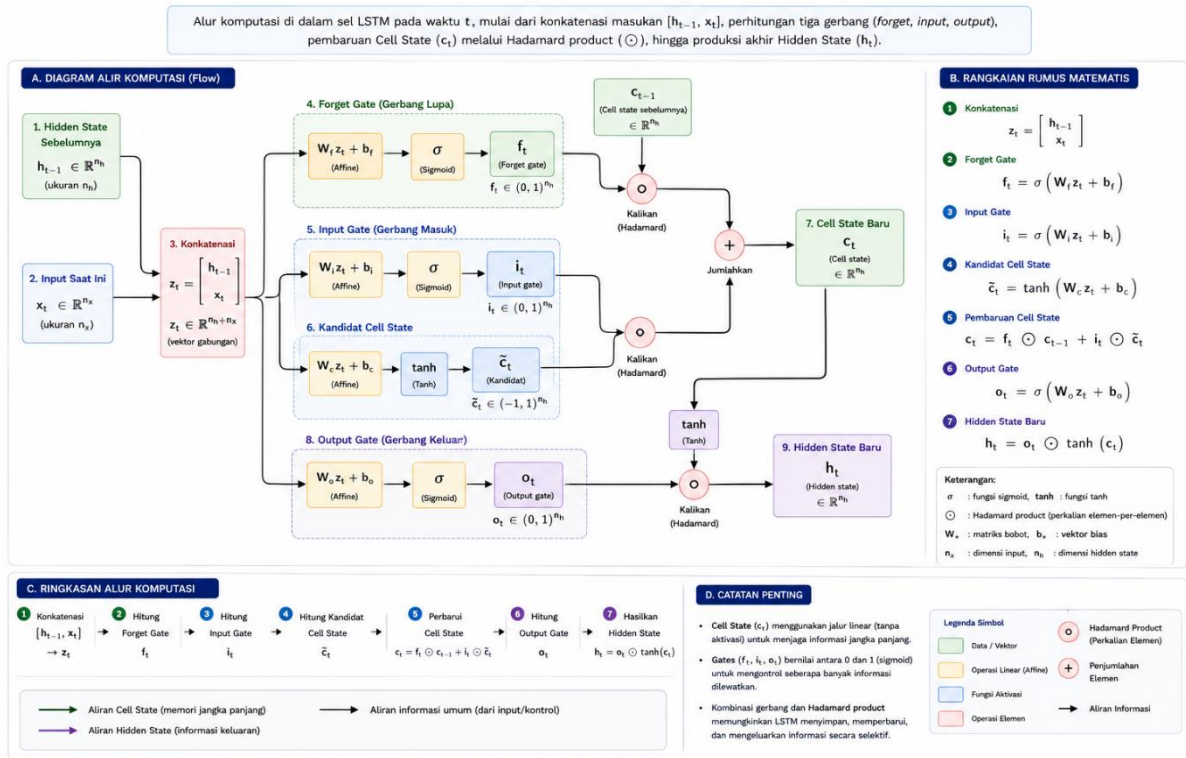
$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

4. Kalkulasi Production Output Gate beserta Agregasi Final Hidden State:

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t \odot \tanh(c_t)$$

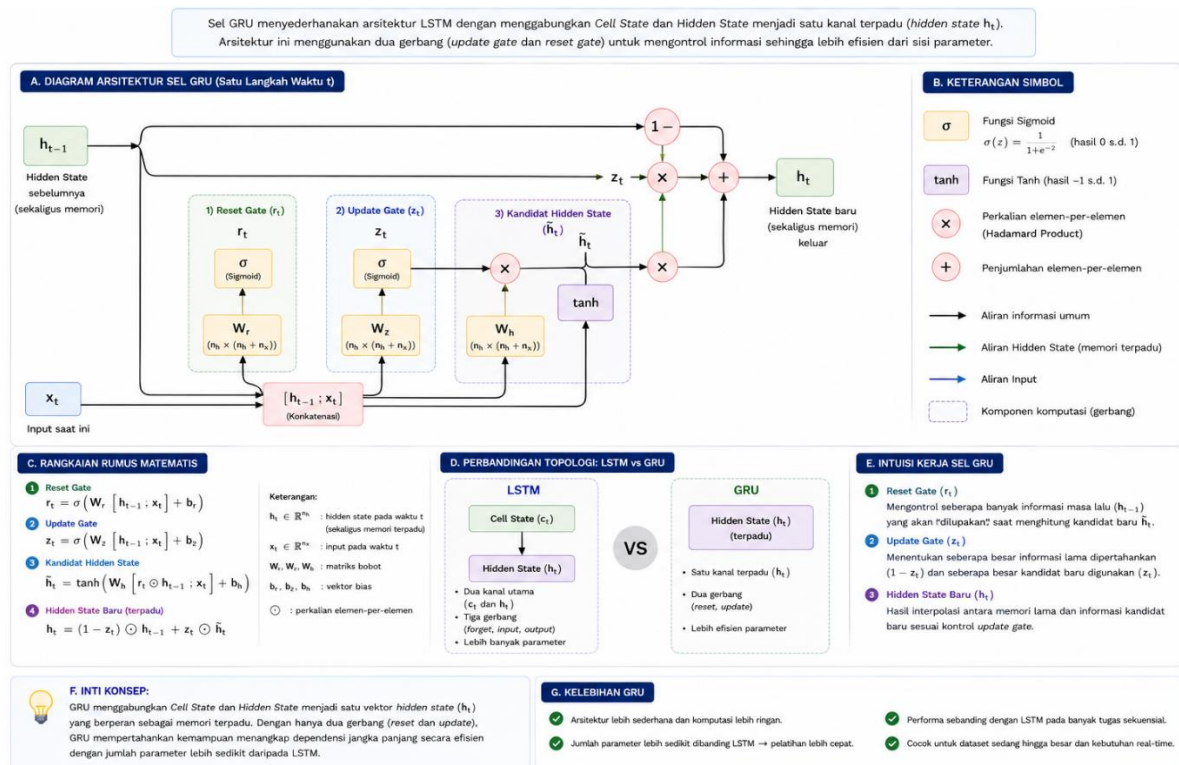
(Operator matematis $\odot$ merepresentasikan perkalian elemen-demi-elemen atau Hadamard product, sementara simbol $[h_{t-1}, x_t]$ merepresentasikan proses penggabungan atau konkatenasi vektor)



Gambar 2.8: Diagram alir komputasi sel LSTM yang memetakan tahapan formulasi matematis, mulai dari konkatenasi vektor $[h_{t-1}, x_t]$, operasi gerbang (σ dan $\tanh$), pembaruan jalur *Cell State* melalui perkalian *Hadamard product* ($\odot$), hingga produksi akhir *Hidden State* (h_t).

2.5 ARSITEKTUR GATED RECURRENT UNIT (GRU)

Untuk menyederhanakan rumitnya topologi kontrol ganda pada LSTM, dikembangkan variasi arsitektur yang lebih minimalis yaitu **Gated Recurrent Unit (GRU)**. Optimalisasi pada GRU dilakukan dengan melebur fungsi *Cell State* seutuhnya ke dalam matriks aliran sekunder *Hidden State*.

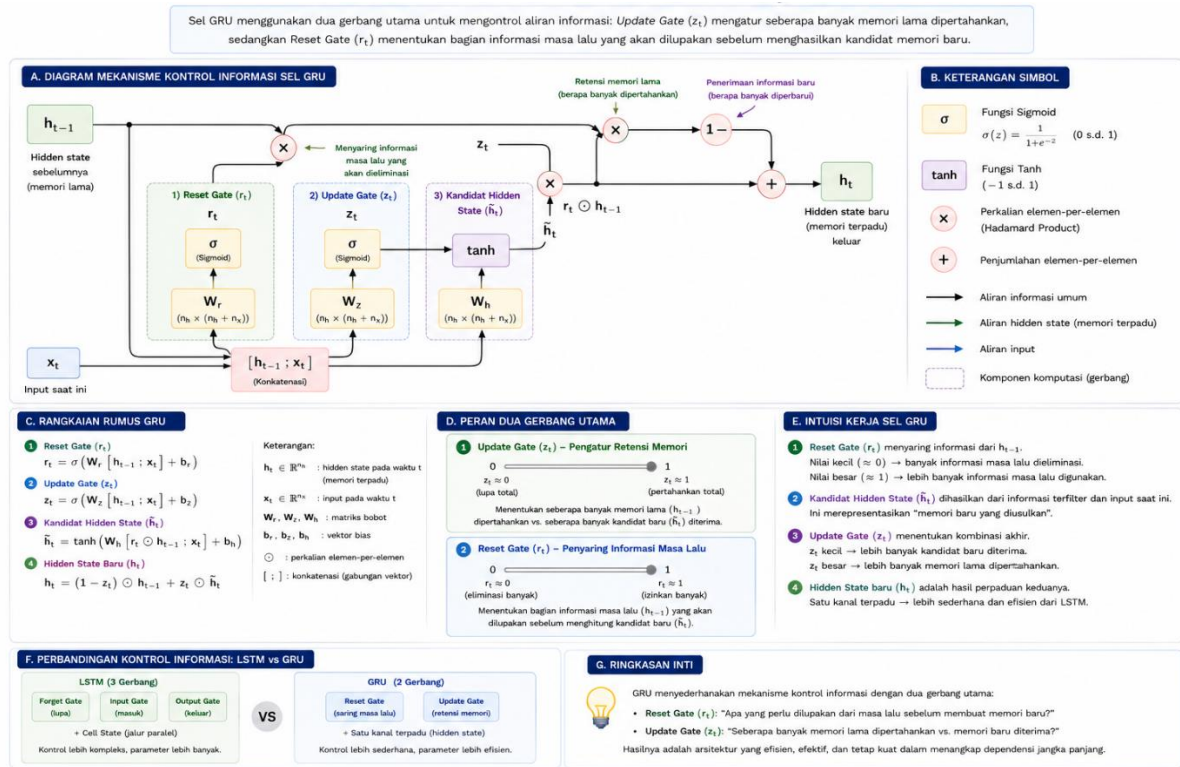


Gambar 2.9: Arsitektur internal sel *Gated Recurrent Unit* (GRU) yang menampilkan penyederhanaan topologi melalui peleburan jalur *Cell State* dan *Hidden State* menjadi satu kanal terpadu untuk efisiensi parameter.

2.5.1 MEKANISME MINIMALIS GATES PADA GRU

Dalam operasional GRU, kendali penyaringan informasi disederhanakan menjadi dua sistem filter:

- Update Gate (z_t):** Merupakan gabungan dari fungsi *forget gate* dan *input gate* pada LSTM. Gerbang ini digunakan untuk menentukan seberapa banyak informasi lama yang dipertahankan dan seberapa banyak informasi baru yang akan ditambahkan.
- Reset Gate (r_t):** Digunakan untuk mengontrol seberapa banyak informasi dari langkah sebelumnya yang akan dilupakan saat menghitung nilai kandidat *hidden state* baru.



Gambar 2.10: Mekanisme kontrol informasi pada sel GRU yang disederhanakan menjadi dua gerbang utama, yaitu *Update Gate* (z_t) sebagai pengatur retensi memori dan *Reset Gate* (r_t) sebagai penyaring informasi masa lalu yang akan dieliminasi.

2.5.2 FORMULASI MATEMATIS INTI GRU

Rangkaian kalkulasi pada GRU dapat disederhanakan ke dalam persamaan berikut:

1. Pengukuran Indeks Proporsi Update Gate:

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t] + b_z)$$

2. Pengukuran Indeks Reduksi Reset Gate:

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t] + b_r)$$

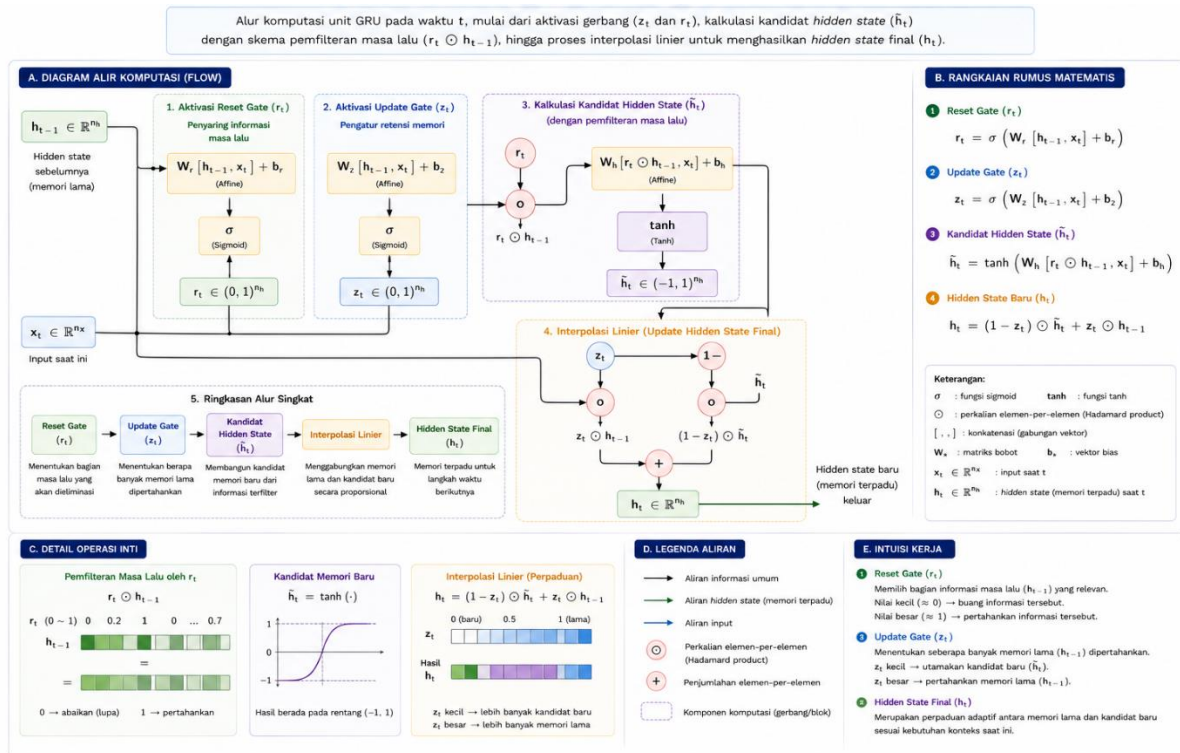
3. Kalkulasi Kandidat Hidden State Internal:

$$\tilde{h}_t = \tanh(W_h \cdot [r_t \odot h_{t-1}, x_t] + b_h)$$

4. Kalkulasi Hidden State Final:

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

Evaluasi Efisiensi GRU: Karena memiliki parameter bobot yang lebih sedikit, model GRU dapat dilatih secara eksponensial lebih cepat dibandingkan dengan topologi LSTM.



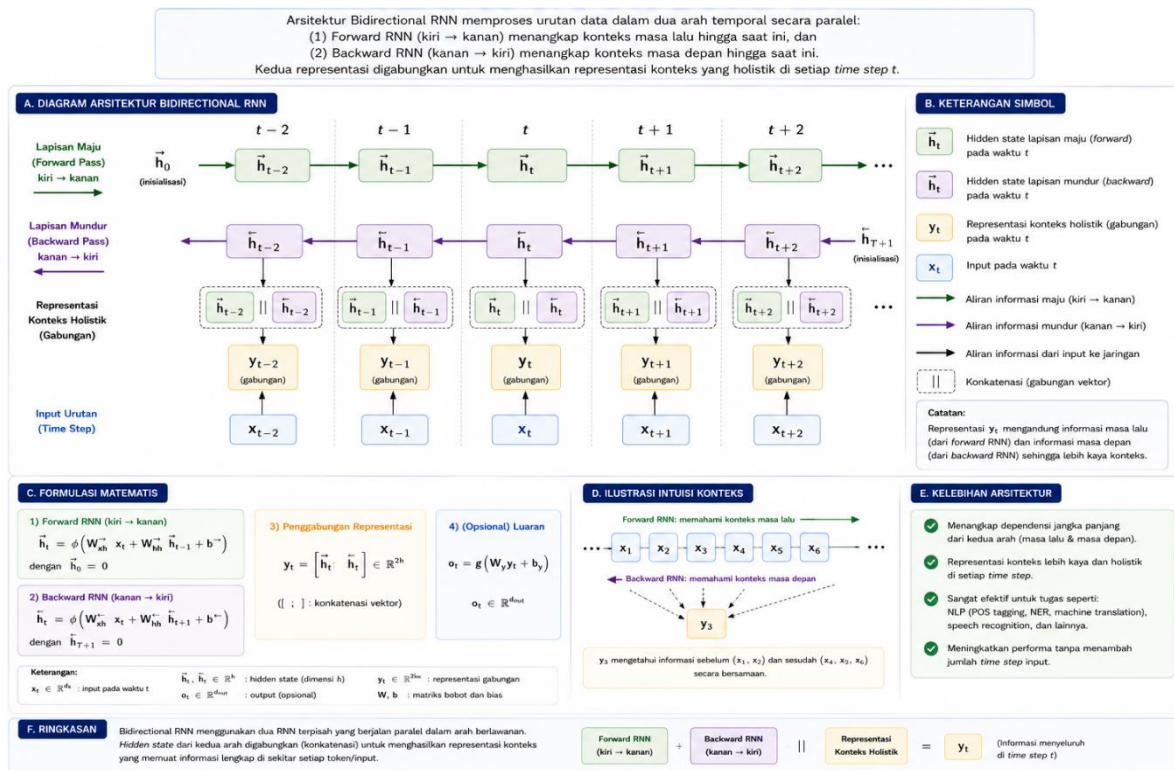
Gambar 2.11: Diagram alir komputasi unit GRU yang memetakan tahapan formulasi matematis inti, mulai dari aktivasi gerbang (z_t dan r_t), kalkulasi kandidat *hidden state* ($\tilde{h}_t$) dengan skema pemfilteran masa lalu ($r_t \odot h_{t-1}$), hingga proses interpolasi linier untuk menghasilkan *Hidden State* final (h_t).

2.6 TEKNIK ARSITEKTURAL EKSTENSIF LANJUTAN (ADVANCE ARCHITECTURES)

Berbagai taktik konseptual arsitektural lazim diimplementasikan untuk mendorong kinerja model mencapai performa maksimum:

2.6.1 TOPOLOGI BIDIRECTIONAL RNN

Arah evaluasi model *vanilla* biasanya dilakukan secara mutlak maju dari kiri ke kanan (*left-to-right*). Skema integrasi **Bidirectional RNN** digunakan untuk memproses sekuens informasi secara paralel dengan dua orientasi pembacaan ganda. Asimilasi konteks informasi dikomputasikan baik secara maju (*forward*) maupun mundur (*backward*). Pendekatan ini sangat krusial dalam bidang analisis kontekstual seperti terjemahan bahasa (*Machine Translation*), di mana keseluruhan kalimat dapat dievaluasi secara utuh.

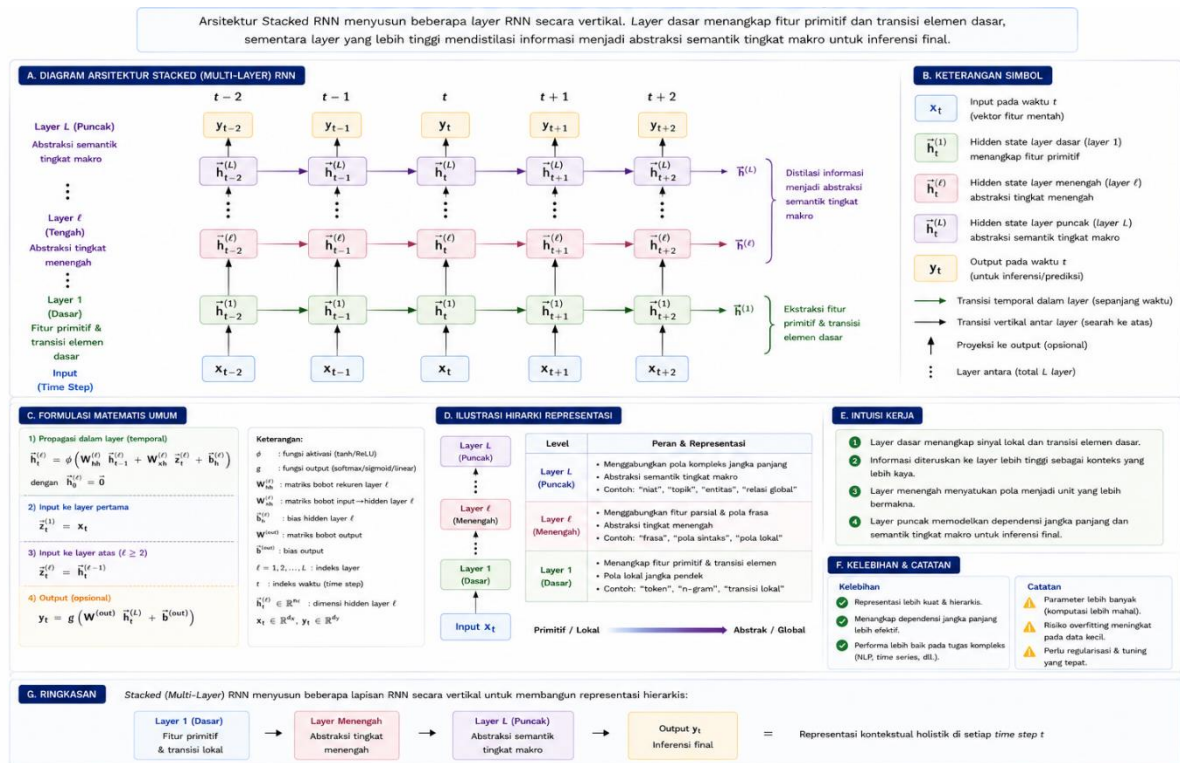


Gambar 2.12: Arsitektur *Bidirectional RNN* yang memperlihatkan pemrosesan paralel dua arah temporal: lapisan maju (*forward pass*) dari kiri ke kanan ($t - 1$ ke $t + 1$) dan lapisan mundur (*backward pass*) dari kanan ke kiri ($t + 1$ ke $t - 1$) untuk menghasilkan representasi konteks yang holistik.

2.6.2 EKSTENSI HIERARKIS STACKED (MULTI-LAYER) RNN

Strategi pelapisan model secara mendalam ini dikenal dengan paradigma **Stacked RNN**. Arsitektur ini difungsikan untuk mendapatkan penyerapan fitur dari tingkatan level yang berbeda:

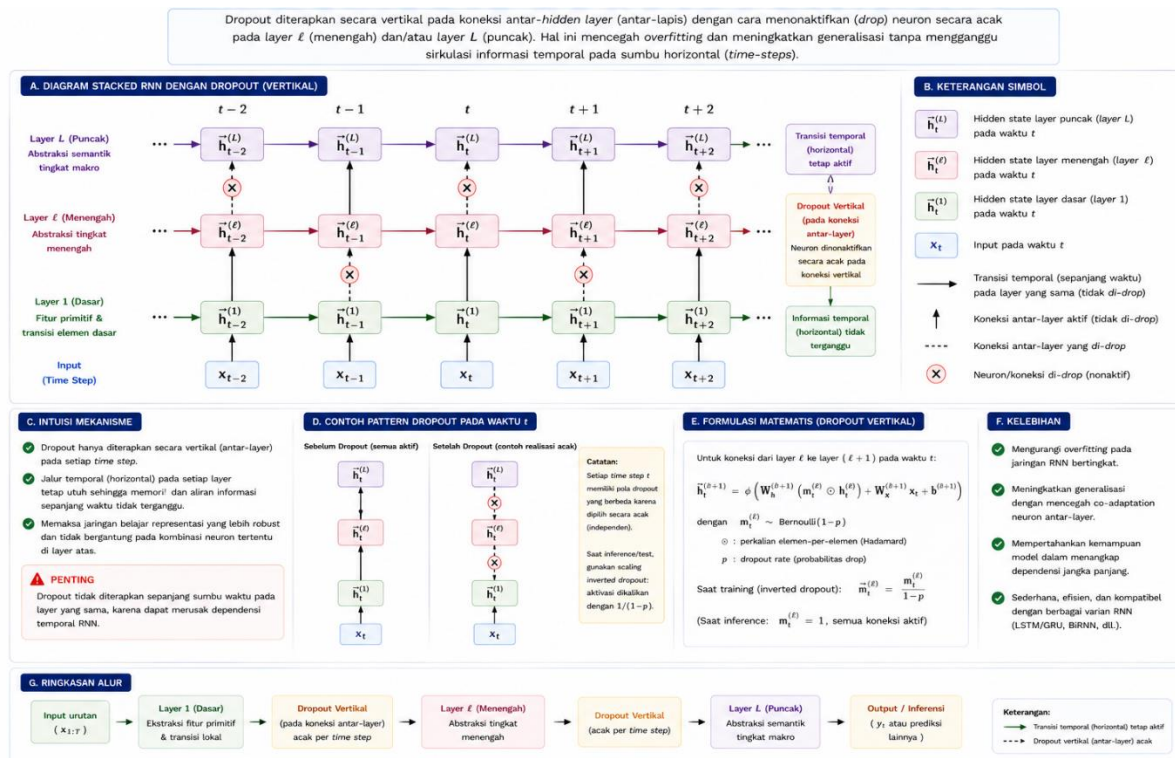
- **Layer Dasar Ekstraksi:** Berupaya melacak pemahaman spesifikasi fisik dari rentang waktu yang berdekatan.
- **Layer Puncak Inferensi:** Memproyeksikan gabungan dari seluruh informasi yang ada untuk memisahkan kompleksitas dan mendukung keputusan komputasi dari unit-unit di bawahnya.



Gambar 2.13: Arsitektur *Stacked* (Multi-Layer) RNN yang menunjukkan susunan sel secara vertikal; *layer* dasar berfungsi mengekstraksi fitur primitif dan transisi elemen dasar, sementara *layer* puncak mendistilasi informasi menjadi abstraksi semantik tingkat makro untuk inferensi final.

2.6.3 INTERVENSI PREVENTIF REGULARISASI DROPOUT

Pengendalian distorsi model akibat masalah *Overfitting* diselesaikan melalui teknik penyisipan sistem redam artifisial, yang dikenal dengan metode **Dropout**. Pada RNN, teknik ini tidak diimplementasikan secara horizontal di sepanjang sumbu rentang waktu (*time-steps*), melainkan digunakan secara spesifik melintang vertikal melewati lapisan antar *hidden layer*.

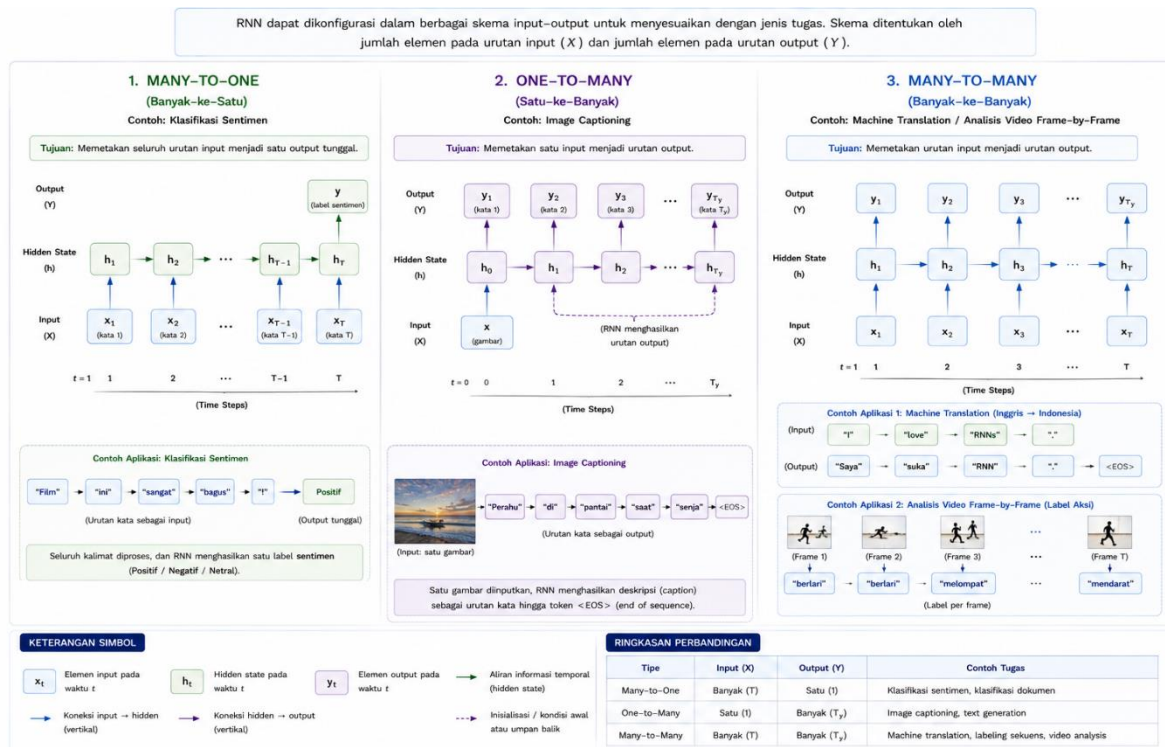


Gambar 2.14: Ilustrasi penerapan regularisasi *Dropout* pada *Stacked RNN*. Penonaktifan neuron secara acak dilakukan secara vertikal pada jalur koneksi antar-*hidden layer* untuk mencegah *overfitting*, tanpa mengganggu sirkulasi informasi temporal pada sumbu horizontal (*time-steps*).

2.7 KATEGORISASI VARIANSI SKEMATIK TIPE INPUT/OUTPUT

Model kognisi temporal RNN bersifat adaptif dalam menyikapi berbagai skema sekuens variabel *input/output*:

- Many-to-One:** Memproyeksikan deretan elemen temporal menjadi pemetaan satu luaran atau prediksi statis final (Contoh: Analisis Klasifikasi Sentimen Teks, Deteksi Anomali).
- One-to-Many:** Memproyeksikan vektor input unit tunggal menjadi luaran berbentuk rentang sekuens (Contoh: Generasi Musik, Deskripsi Otomatis Citra (*Image Captioning*)).
- Many-to-Many:** Melaksanakan mekanisme transformasi dari barisan masukan menjadi barisan sekuens keluaran dinamis secara asinkron atau sinkron (Contoh: Terjemahan Bahasa (*Seq2seq*), Analisis *Frame-by-Frame* pada Video).



Gambar 2.15: Variansi skematik tipe *input-output* pada RNN, mengilustrasikan konfigurasi *Many-to-One* (misal: klasifikasi sentimen), *One-to-Many* (misal: *image captioning*), dan *Many-to-Many* (misal: *machine translation* atau analisis video *frame-by-frame*).

DEPENDENSI

CODE

```
# --- Core ---
import sys
import os
import math

# --- Numerik & Array ---
import numpy as np

# --- Image Processing ---
from PIL import Image, ImageDraw, ImageFont, ImageFilter, ImageEnhance
import cv2

# --- Visualisasi ---
import matplotlib.pyplot as plt
import matplotlib.patches as patches

# --- Metrik Kualitas Gambar ---
from skimage.metrics import structural_similarity as ssim
```

CODE

```
# Memastikan folder output ada
os.makedirs("output", exist_ok=True)
```